BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH – VIỄN THÔNG

BÁO CÁO LAB

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

THIẾT KẾ BỘ XỬ LÝ RISC-V 32BIT

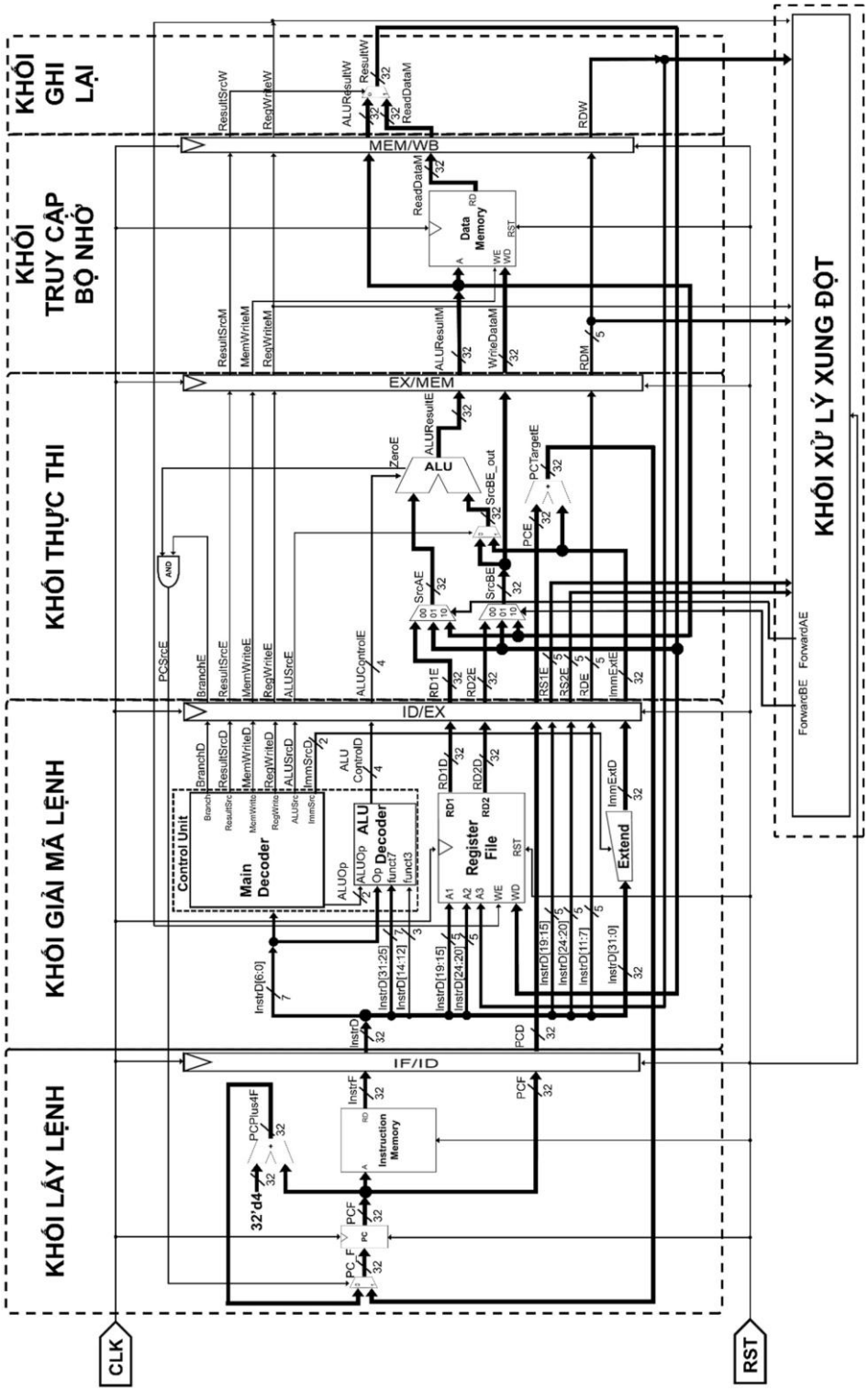
GVHD: TS. Đỗ Duy Tân

SVTH: Phạm Quang Hợp

MSSV: 22119178

TP. Hồ Chí Minh, tháng 3 năm 2025

1. Hoạt động hệ thống



Hình 1: Sơ đồ chi tiết hệ thống

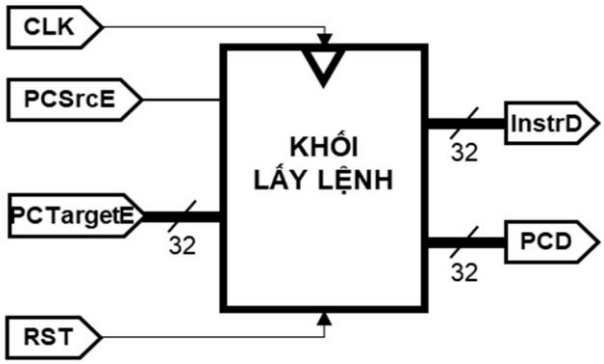
Hệ thống CPU 32 bit RISC-V sử dụng kỹ thuật đường ống 5 tầng để tối ưu hóa hiệu suất và tốc độ xử lý bằng cách chia quá trình thực hiện lệnh thành năm giai đoạn riêng biệt: lấy lệnh, giải mã lệnh, thực thi, truy cập bộ nhớ, và ghi lại.

Trong giai đoạn lấy lệnh, bộ nhớ lệnh cung cấp các lệnh từ địa chỉ do PC chỉ định, và PC được cập nhật để trỏ đến lệnh tiếp theo. Giai đoạn giải mã lệnh giải mã lệnh để xác định các thanh ghi nguồn, thanh ghi đích, và loại thao tác, đồng thời đọc giá trị từ các thanh ghi nguồn và tạo ra các tín hiệu điều khiển cần thiết. Ở giai đoạn thực thi, ALU thực hiện các thao tác tính toán hoặc luận lý dựa trên các giá trị đầu vào và tín hiệu điều khiển, và kết quả được lưu vào thanh ghi trung gian EX/MEM. Giai đoạn truy cập bộ nhớ thực hiện việc đọc hoặc ghi dữ liệu từ hoặc đến bộ nhớ dữ liệu nếu lệnh yêu cầu, và lưu kết quả vào thanh ghi MEM/WB. Cuối cùng, giai đoạn ghi lại ghi kết quả cuối cùng của lệnh vào tệp thanh ghi, cập nhật giá trị của thanh ghi đích.

Hệ thống sử dụng các khối thanh ghi trung gian giữa các giai đoạn để lưu trữ thông tin tạm thời và các đơn vị điều khiển để quản lý dòng chảy dữ liệu và điều khiển các tín hiệu. Khối xử lý xung đột đảm bảo các xung đột dữ liệu và điều khiển được xử lý một cách hiệu quả, sử dụng các kỹ thuật chuyển tiếp dữ liệu. Nhờ vào thiết kế này, hệ thống có thể thực thi nhiều lệnh cùng lúc, tăng cường hiệu suất tổng thể.

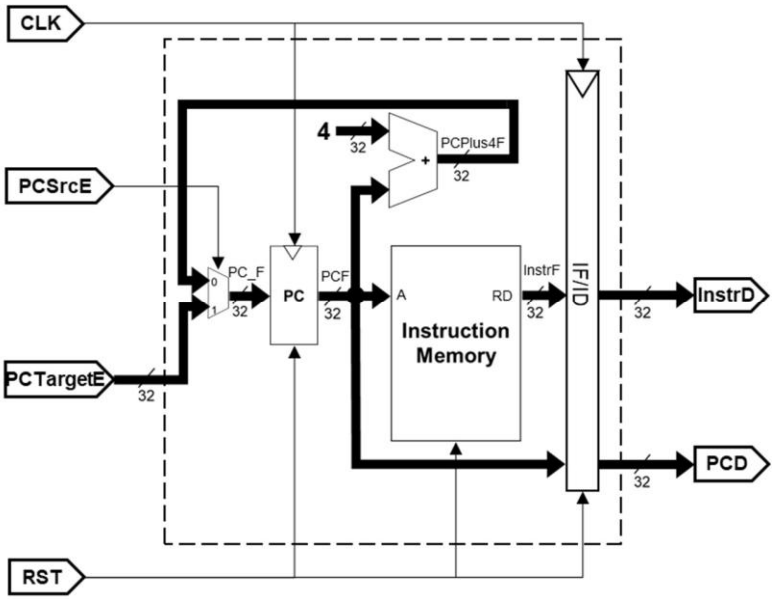
2. Khối lấy lệnh

Sơ đồ khối: gồm 4 ngõ vào và 2 ngõ ra.



Hình 2: Sơ đồ Khối lấy lệnh

Sơ đồ chi tiết gồm các mô đun: bộ ghép kênh chọn đầu vào của chương trình, bộ cộng, bộ đếm chương trình, bộ nhớ lệnh và thanh ghi IF/ID.



Hình 3: Sơ đồ chi tiết Khối lấy lệnh

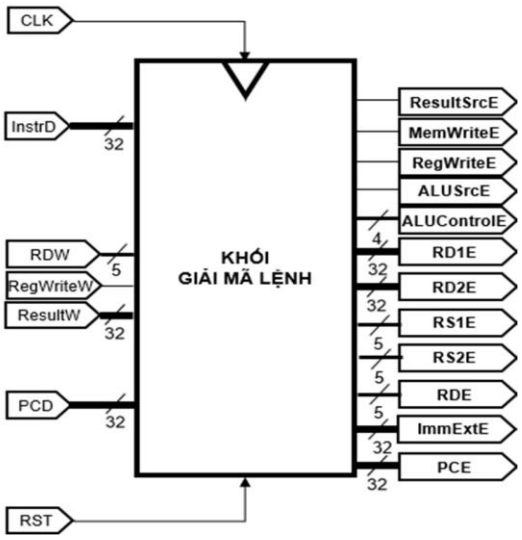
Khối lấy lệnh trong Hình 3 thực hiện nhiệm vụ lấy lệnh từ bộ nhớ lệnh và cập nhật bộ đếm chương trình. Dưới đây là các bước hoạt động chi tiết của khối lấy lệnh:

- Tín hiệu PCSrcE và PCTargetE:
  - PCSrcE: Tín hiệu điều khiển, quyết định nguồn giá trị mới của PC.
  - PCTargetE: Địa chỉ đích, được sử dụng khi PCSrcE = 1.
  - PCPlus4F: Giá trị của PC được cộng thêm 4, sử dụng để trỏ đến lệnh kế tiếp trong bộ nhớ.
- Bộ ghép kênh chọn đầu vào cho PC.
  - Nếu PCSrcE = 0, chọn PC + 4 (tăng PC lên 4 để trỏ đến lệnh kế tiếp).
  - Nếu PCSrcE = 1, chọn giá trị từ PCTargetE.
- Bộ đếm chương trình lưu trữ giá trị hiện tại của bộ đếm chương trình. Giá trị này được cập nhật mỗi chu kỳ xung đồng hồ.
- Bộ nhớ lệnh, lưu trữ các lệnh của chương trình. Địa chỉ A là giá trị của PC. Đầu ra RD (InstrF) là lệnh tại địa chỉ tương ứng.

- Thanh ghi IF/ID là thanh ghi đường ống, lưu trữ lệnh (InstrF) và giá trị PC (PCF) để chuyển sang giai đoạn giải mã lệnh (InstrD và PCD).

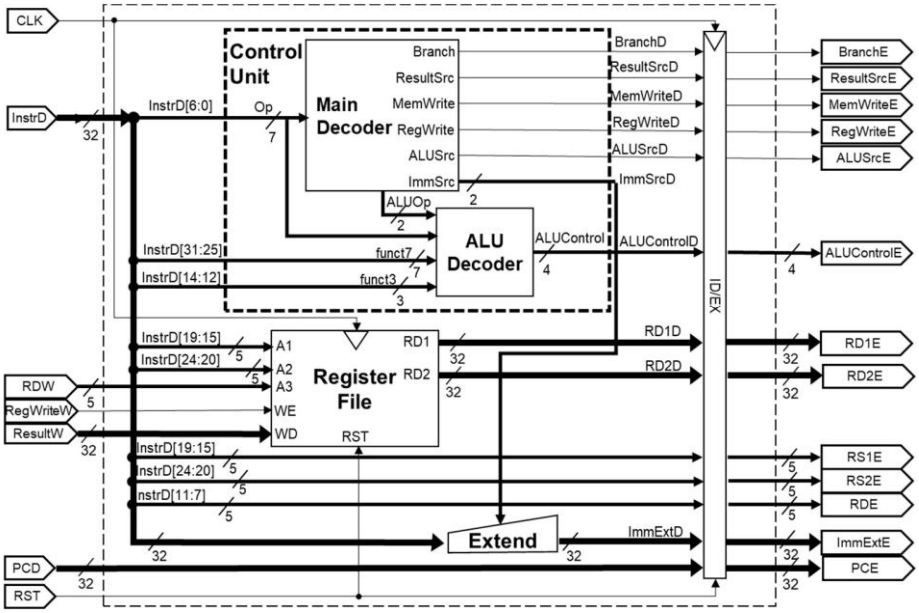
3. Khối giải mã lệnh

Sơ đồ khối: gồm 7 ngõ vào và 13 ngõ ra.



Hình 4: Sơ đồ Khối giải mã lệnh

Sơ đồ chi tiết gồm các mô đun: Đơn vị điều khiển có bộ giải mã chính và bộ giải mã ALU, tệp thanh ghi, khối mở rộng và thanh ghi ID/EX.



Hình 5: Sơ đồ chi tiết Khối giải mã lệnh

Khối giải mã lệnh ở Hình 5 có nhiệm vụ phân tích lệnh và tạo ra các tín hiệu điều khiển cần thiết để thực hiện lệnh. Quá trình giải mã bao gồm việc xác định loại lệnh, các tham số của lệnh, và điều khiển các đơn vị chức năng khác trong CPU. Dưới đây là mô tả chi tiết các bước hoạt động của khối giải mã lệnh:

- Bộ giải mã chính gồm tín hiệu đầu vào là Op nhận 7 bit opcode từ lệnh (InstrD[6:0]) để xác định loại lệnh và tạo ra các tín hiệu điều khiển như mô tả tại Bảng:
  - Branch: Tín hiệu nhánh.
  - ResultSrc: Chọn nguồn kết quả (ALU hoặc bộ nhớ).
  - MemWrite: Tín hiệu ghi bộ nhớ.
  - RegWrite: Tín hiệu ghi thanh ghi.
  - ALUSrc: Chọn đầu vào thứ hai của ALU (thanh ghi hoặc hằng số mở rộng). ImmSrc: Chọn loại hằng số mở rộng.
- Bộ giải mã ALU có tín hiệu đầu vào funct7, funct3 và ALUOp nhận các bit chức năng từ lệnh (InstrD[31:25], InstrD[14:12]) để tạo ra tín hiệu điều khiển ALU(ALUControlD) như mô tả Bảng 1
- Bộ thanh ghi:
  - A1, A2, A3: Địa chỉ các thanh ghi nguồn và đích từ lệnh (InstrD[19:15], InstrD[24:20], InstrD[11:7]).
  - RD1, RD2: Giá trị đọc từ các thanh ghi nguồn.
  - WD: Giá trị ghi vào thanh ghi đích khi thực hiện lệnh ghi.
  - WE: Tín hiệu cho phép ghi thanh ghi.
- Khối mở rộng hằng số từ lệnh, được tạo ra từ các bit tương ứng của lệnh như Bảng 3
- Thanh ghi ID/EX lưu trữ các tín hiệu điều khiển và giá trị cần thiết để chuyển sang giai đoạn thực thi lệnh.

Bảng 1: Bảng trạng thái của tín hiệu điều khiển với từng loại lệnh

Instruction	**Op**	**RegWrite**	**ImmSrc**	**ALUSrc**	**MemWrite**	**ResultSrc**	**Branch**	**ALUOp**
I-Type (loads)	0000011	1	00	1	0	1	0	00
S-Type (Stores)	0100011	0	01	1	1	x	0	00
B-Type (Branch)	1100011	0	10	0	0	x	1	01
I-Type (Arithmetic)	0010011	1	00	1	0	0	0	00
R-Type	0110011	1	xx	0	0	0	0	10

Bảng 2: Bảng trạng thái của tín hiệu điều khiển ALU

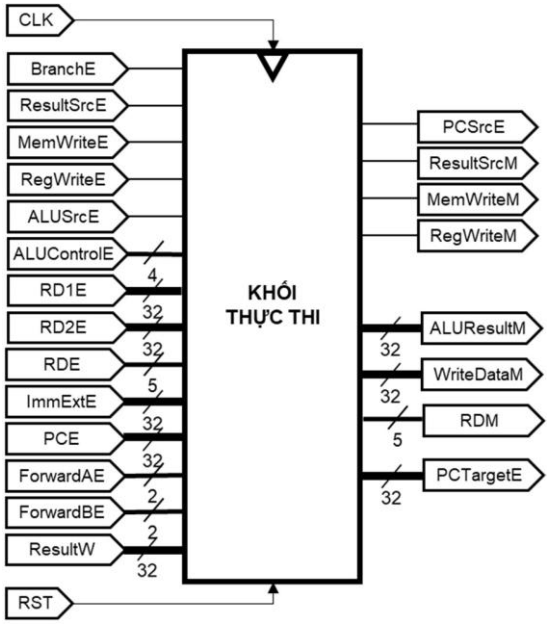
ALUOP	**funct3**	**{op[5],funct7[5],funct7[0]}**	**ALUControl**	**Instruction**
00	x	x	0000	lw,sw
01	x	x	0001	beq
10	000	000,001,010,011,100,111	0000	add
		110	0001	sub
		101	0010	mul
	001	x	0011	sll
	010	x	0100	slt
	100	xx0	0101	xor
		xx1	0110	div
	101	x0x	0111	srl
		x1x	1000	sra
	110	xx0	1001	or
		xx1	1010	rem
	111	x	1011	and

Bảng 3: Mã hóa ImmSrc

ImmSrc	ImmExt	Loại	Mô tả
00	{{20{Instr[31]}}, Instr[31:20]}	I	Giá trị tức thời 12 bit có dấu
01	{{20{Instr[31]}},Instr[31:20],Instr[11:7]}	S	Giá trị tức thời 12 bit có dấu
10	{{20{Instr[31]}},Instr[7],Instr[30:25],Instr[11:8],1'b0}	B	Giá trị tức thời 13 bit có dấu

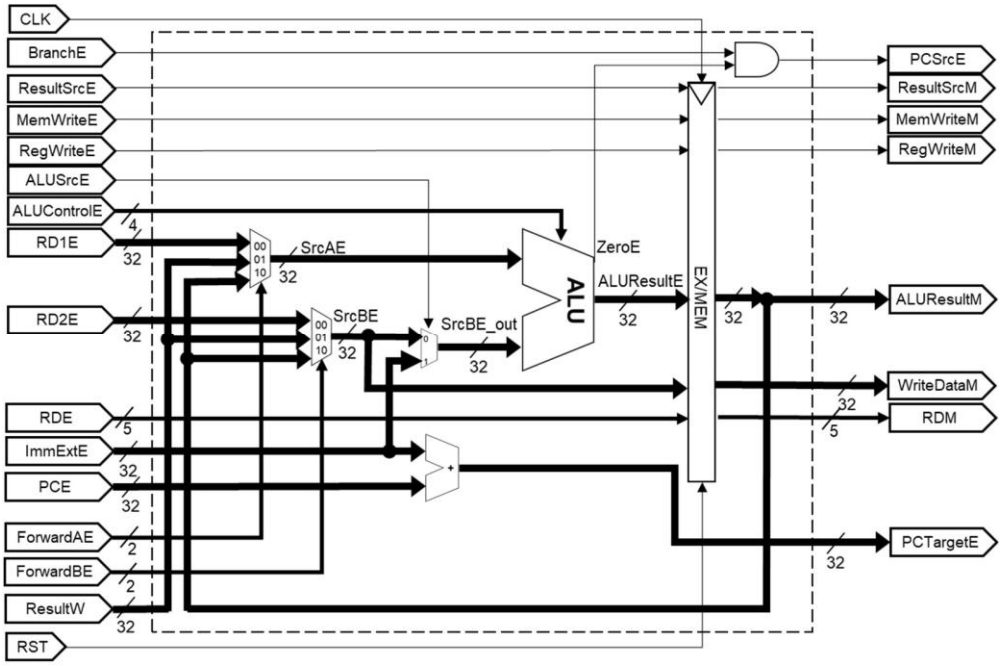
4. Khối thực thi

Sơ đồ khối: gồm 16 ngõ vào và 8 ngõ ra.



Hình 6: Sơ đồ Khối thực thi

Sơ đồ chi tiết gồm: Đơn vị luận lý số học, bộ cộng, bộ ghép kênh, thanh ghi EX/MEM.



Hình 7: Sơ đồ chi tiết Khối thực thi

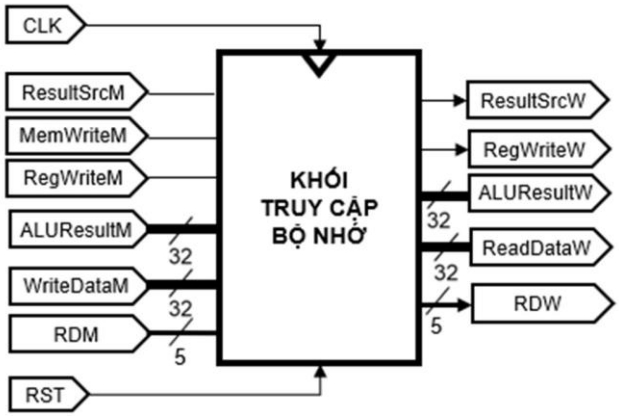
Khối thực thi ở Hình 7 có chức năng chính là thực hiện các phép tính và hoạt động được xác định bởi lệnh, như được giải mã từ khối giải mã lệnh. Chức năng cụ thể của khối thực thi bao gồm:

- ALU thực hiện các phép toán số học và luận lý dựa trên tín hiệu điều khiển ALUControlE. Các giá trị đầu vào cho ALU được chọn từ thanh ghi, giá trị tức thời hoặc các giá trị chuyển tiếp. Tín hiệu ZeroE chỉ ra kết quả ALU bằng không, dùng cho các quyết định nhánh.
- Xử lý xung đột: Các tín hiệu ForwardAE và ForwardBE xử lý xung đột dữ liệu bằng cách chuyển tiếp dữ liệu từ các giai đoạn đường ống sau về giai đoạn thực thi.
- Các bộ chọn lựa chọn giữa dữ liệu chuyển tiếp, dữ liệu thanh ghi hoặc các giá trị tức thời cho các phép tính của ALU.
- Tín hiệu điều khiển: BranchE, ResultSrcE, MemWriteE, RegWriteE, ALUSrcE, ALUControlE điều khiển các hành động của khối thực thi. Tín hiệu ResultW điều khiển kết quả cuối cùng.

- Tín hiệu đầu ra ALUResultM chuyển kết quả từ ALU được chuyển đến giai đoạn bộ nhớ. Tín hiệu WriteDataM ghi dữ liệu vào bộ nhớ. Tín hiệu PCTargetE địa chỉ mục tiêu cho các lệnh nhánh.

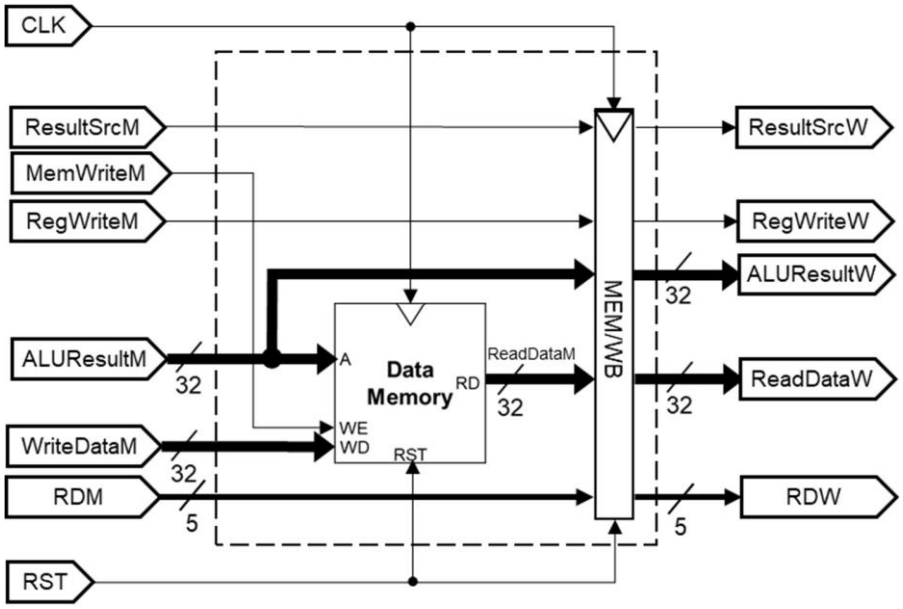
5. Khối truy cập bộ nhớ

Sơ đồ khối bao gồm có 8 ngõ vào và 5 ngõ ra:



Hình 8: Sơ đồ Khối truy cập bộ nhớ

Sơ đồ chi tiết gồm bộ nhớ dữ liệu và thanh ghi MEM/WB



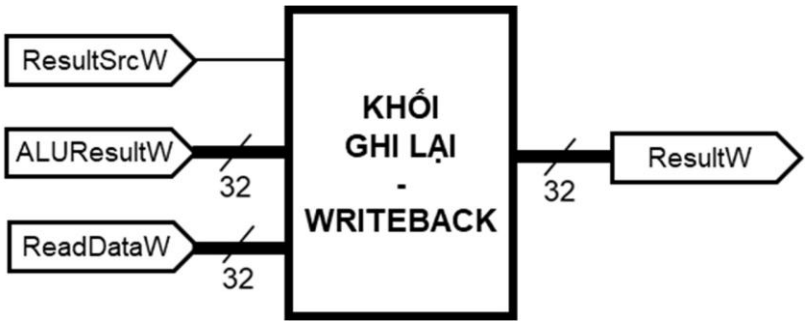
Hình 9: Sơ đồ chi tiết Khối truy cập bộ nhớ

Khối truy cập bộ nhớ đọc và ghi dữ liệu vào bộ nhớ chính. Khối truy cập bộ nhớ có các chức năng quan trọng sau:

- Tín hiệu MemWriteM điều khiển cho phép ghi dữ liệu vào bộ nhớ.
- Tín hiệu ALUResultM là địa chỉ từ đơn vị tính toán (ALU) để ghi vào bộ nhớ.
- Tín hiệu WriteDataM là dữ liệu để ghi vào bộ nhớ.
- Thanh ghi MEM/WB lưu trữ các tín hiệu điều khiển và giá trị cần thiết để chuyển sang giai đoạn thực thi lệnh.

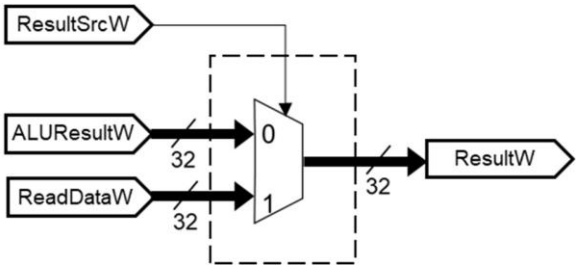
6. Khối ghi lại

Sơ đồ khối: gồm 3 ngõ vào và 1 ngõ ra.



Hình 10: Sơ đồ Khối ghi lại

Sơ đồ chi tiết chỉ có một bộ ghép kênh 2 ngõ vào.



Hình 11: Sơ đồ chi tiết Khối ghi lại

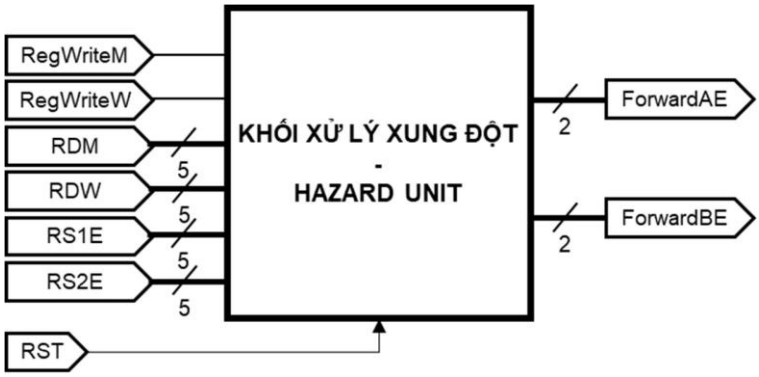
Khối ghi lại trong sơ đồ này có chức năng chọn kết quả cuối cùng để ghi vào các thanh ghi trong hệ thống. Hoạt động của khối này có thể được giải thích như sau:

- Tín hiệu ResultSrcW điều khiển để chọn nguồn dữ liệu kết quả. Tín hiệu ALUResultW là dữ liệu kết quả từ đơn vị tính toán (ALU). Tín hiệu ReadDataW là dữ liệu được đọc từ bộ nhớ.

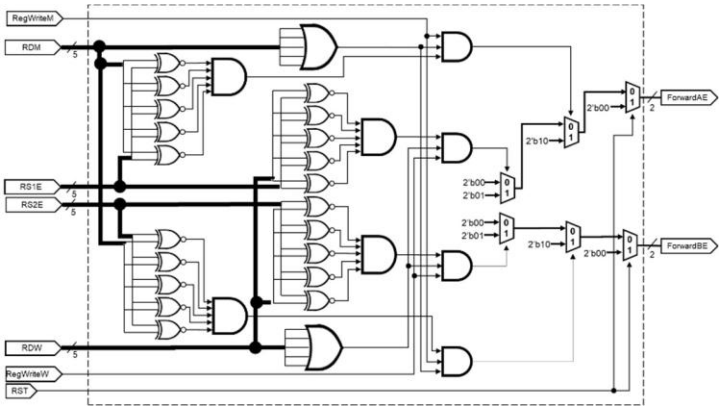
- Bộ chọn quyết định dữ liệu nào sẽ được truyền ra ngoài thông qua tín hiệu điều khiển ResultSrcW.

7. Khối xử lý xung đột

Khối xử lý xung đột trong sơ đồ này có nhiệm vụ phát hiện và giải quyết các xung đột xảy ra trong quá trình thực hiện lệnh.



Hình 12: Sơ đồ Khối xử lý xung đột



Hình 13: Sơ đồ chi tiết Khối xử lý xung đột

ForwardAE, ForwardBE được sử dụng để chọn nguồn dữ liệu phù hợp để tránh xung đột:

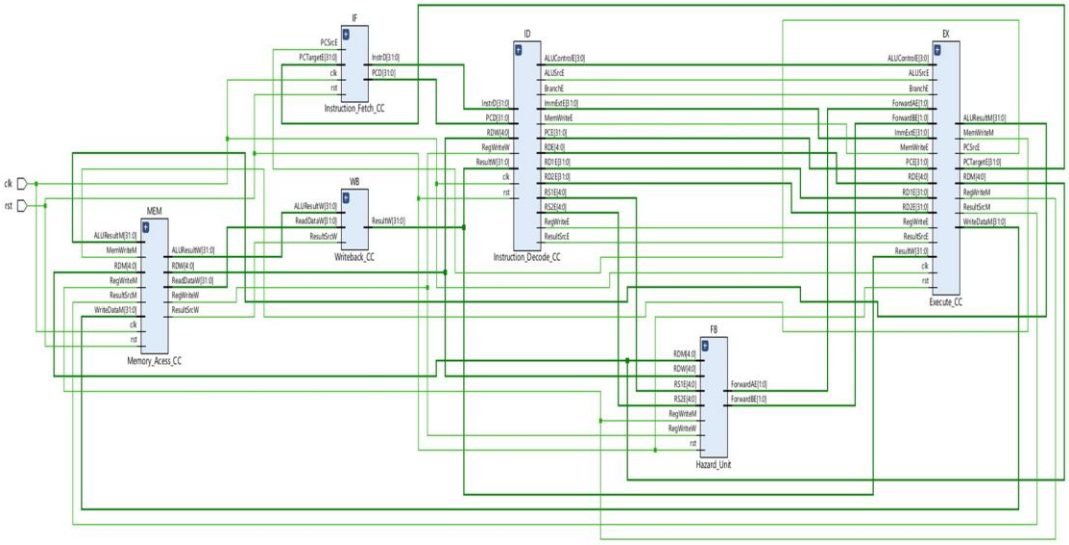
- Nếu có xung đột, khối sẽ điều khiển để chuyển tiếp dữ liệu từ giai đoạn trước đến giai đoạn hiện tại , đảm bảo rằng lệnh hiện tại có được dữ liệu chính xác mà không cần chờ đợi.
- Nếu không có xung đột, các tín hiệu này sẽ cho phép dữ liệu được sử dụng trực tiếp từ các thanh ghi nguồn.

Bảng 4: Các giá trị điều khiển cho các bộ ghép kênh chuyển tiếp

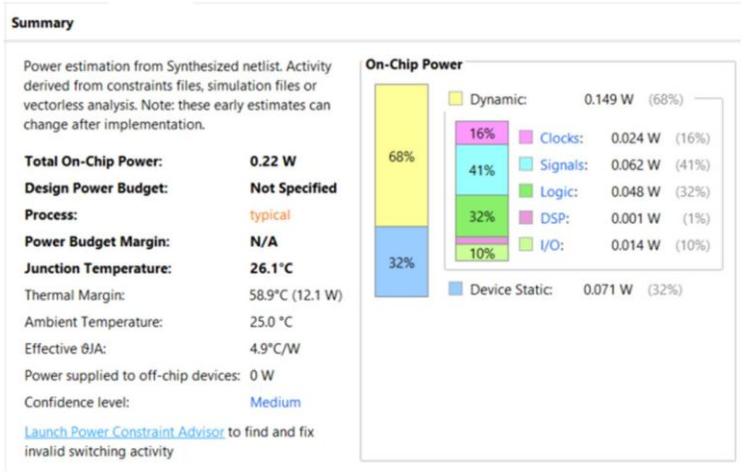
Điều khiển MUX	Nguồn	Giải thích
ForwardA = 00	ID/EX	Toán hạng ALU đầu tiên xuất phát từ Tệp thanh ghi.
ForwardA = 10	EX/MEM	Toán hạng ALU đầu tiên xuất phát từ kết quả ALU trước đó.
ForwardA = 01	MEM/WB	Toán hạng ALU đầu tiên xuất phát từ bộ nhớ dữ liệu.
ForwardB = 00	ID/EX	Toán hạng ALU thứ hai xuất phát từ Tệp thanh ghi.
ForwardB = 10	EX/MEM	Toán hạng ALU thứ hai xuất phát từ kết quả ALU trước đó.
ForwardB = 01	MEM/WB	Toán hạng ALU thứ hai xuất phát từ bộ nhớ dữ liệu.

8. KẾT QUẢ MÔ PHỎNG TOÀN HỆ THỐNG

Sau khi hoàn thành việc thiết kế các khối thành phần, nhóm đã tiến hành kết nối đường dây để được ra sơ đồ RTL của bộ vi xử lý 32 bit hoàn chỉnh như Hình 14 mô tả các khối chức năng chính và cách chúng tương tác trong quá trình thực thi lệnh. Đầu tiên, khối lấy lệnh sẽ lấy lệnh từ bộ nhớ lệnh. Lệnh này sau đó được gửi đến khối giải mã để giải mã và tạo ra các tín hiệu điều khiển cần thiết. Các tín hiệu này được truyền đến khối thực thi, nơi các phép toán số học hoặc logic được thực hiện dựa trên lệnh và các tín hiệu điều khiển. Nếu lệnh yêu cầu truy cập bộ nhớ, khối truy cập bộ nhớ sẽ thực hiện các hoạt động đọc hoặc ghi dữ liệu từ bộ nhớ dữ liệu. Cuối cùng, kết quả của lệnh được ghi vào các thanh ghi thông qua khối ghi kết quả. Sơ đồ này cho thấy một cách rõ ràng các kết nối giữa các khối, minh họa cho quá trình sử dụng kỹ thuật đường ống, nơi nhiều lệnh được thực hiện đồng thời qua các giai đoạn khác nhau, tận dụng kiến trúc Harvard với bộ nhớ lệnh và dữ liệu tách biệt để tăng hiệu suất xử lý. Sơ đồ RTL cũng cho thấy trong việc quản lý luồng dữ liệu và tín hiệu điều khiển giữa các khối, đảm bảo rằng mỗi lệnh được thực thi một cách chính xác và hiệu quả. Với thiết kế này, CPU có thể thực hiện các tác vụ phức tạp một cách nhanh chóng, đáp ứng các yêu cầu của hệ thống máy tính hiện đại.



Hình 14: Sơ đồ RTL của Vi xử lý 32 bit



Hình 15: Công suất hoạt động của Bộ vi xử lý 32 bit

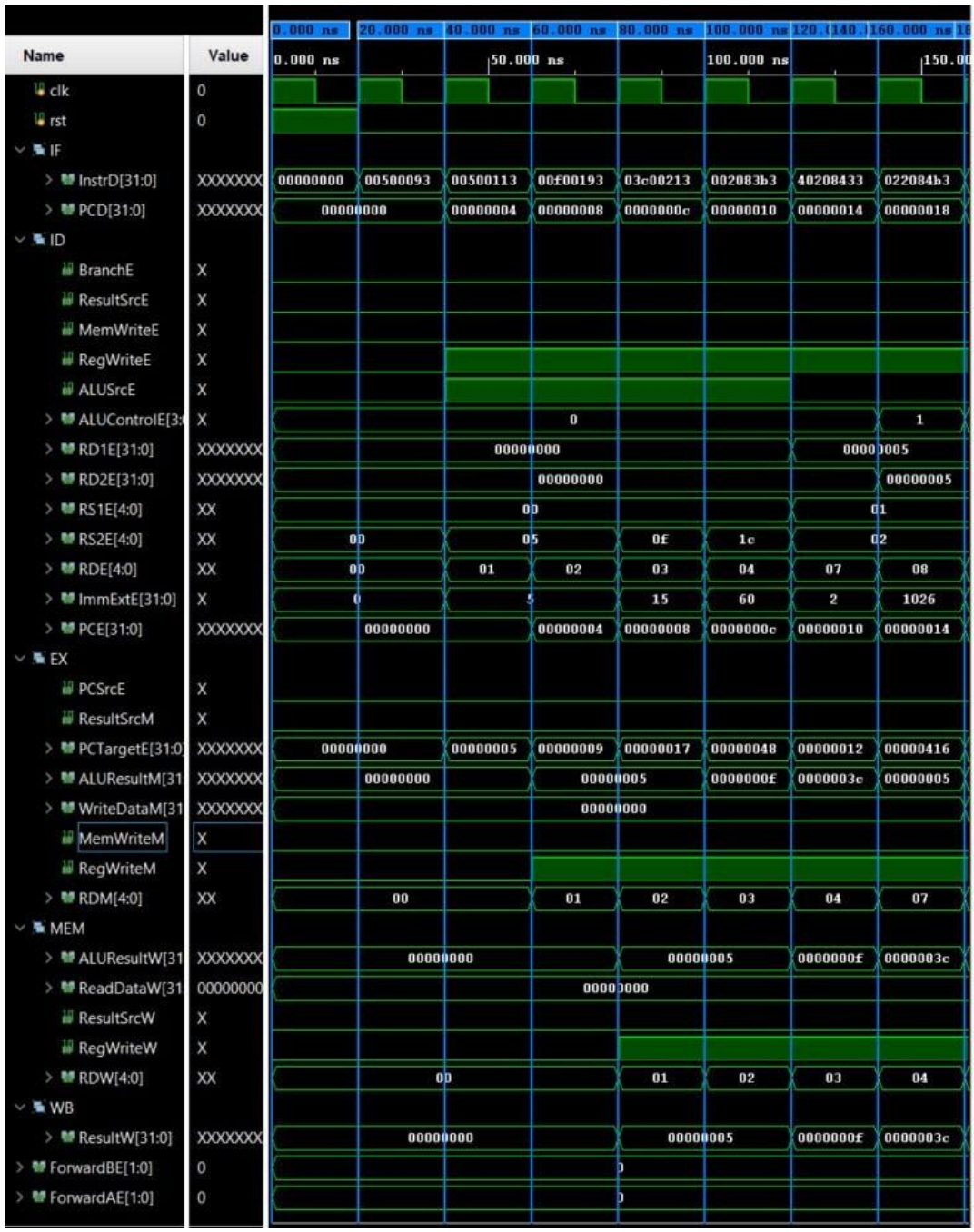
Hình 15 cung cấp một bản tóm tắt về ước tính tiêu thụ điện năng của bộ vi xử lý 32 bit. Tổng công suất tiêu thụ trên chip là 0.22 W, nhiệt độ điểm nối là 26.1°C và biên nhiệt là 58.9°C (12.1 W). Nhiệt độ môi trường là 25.0°C với hiệu quả tản nhiệt là 4.9°C/W. Công suất tiêu thụ trên chip được chia thành hai phần chính: công suất động và công suất tĩnh. Công suất động là 0.149 W, chiếm 68% tổng công suất. Trong đó, các đồng hồ (Clocks) tiêu thụ 0.024 W (16%), các tín hiệu (Signals) tiêu thụ 0.062 W (41%), logic tiêu thụ 0.048 W (32%), DSP tiêu thụ 0.001 W (1%), và I/O tiêu thụ 0.014 W (10%). Công suất tĩnh của thiết bị là 0.071 W, chiếm 32% tổng công suất.

Bảng 5: Mã Assembly dùng mô phỏng chương trình

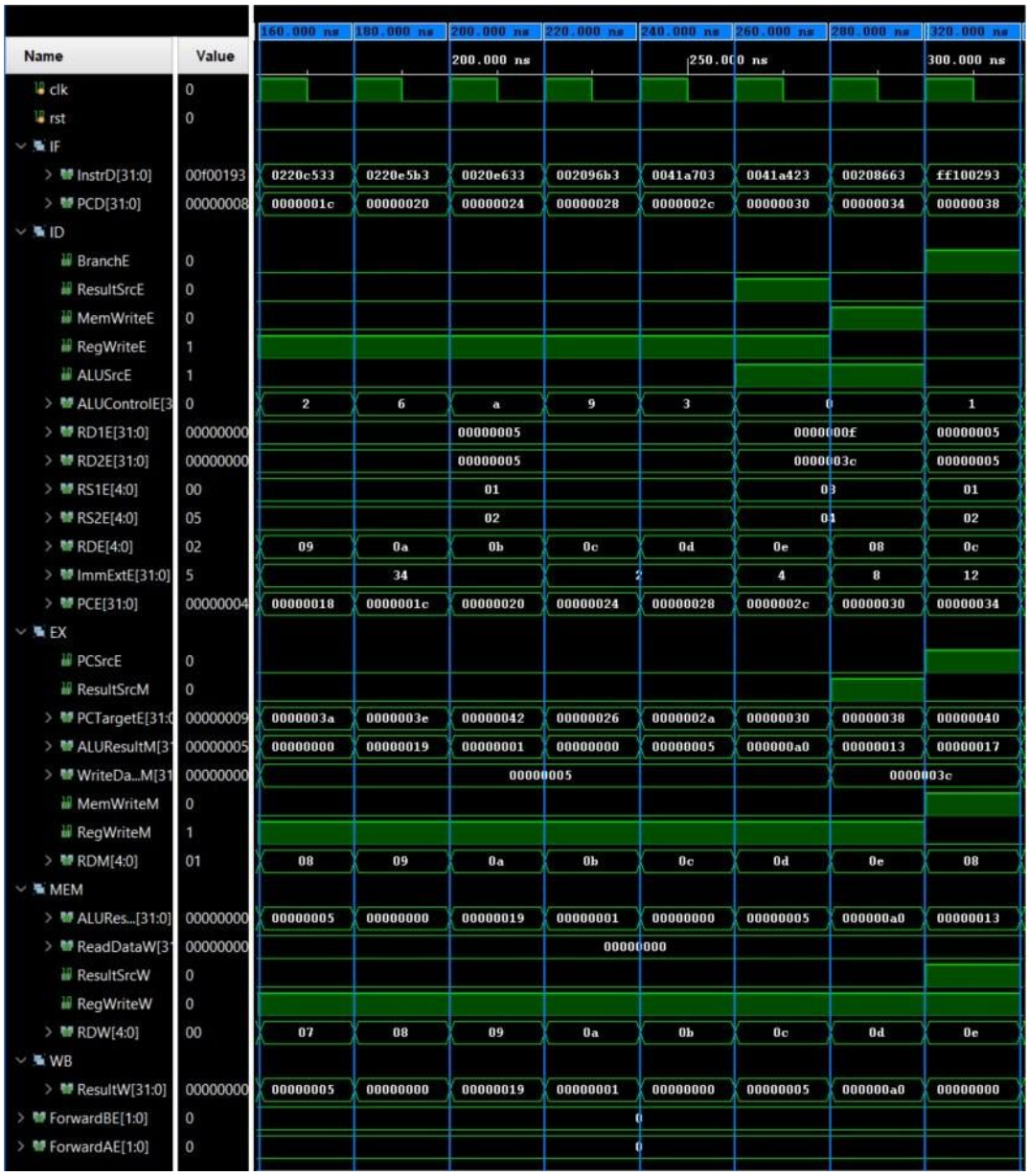
Nhãn	Mã Assembly	Mã HEX
	addi x1, x0, 5	0x00500093
	addi x2, x0, 5	0x00500113
	addi x3, x0, 15	0x00F00193
	addi x4, x0, 60	0x03C00213
label:	add x7, x1, x2	0x002083B3
	sub x8, x1, x2	0x40208433
	mul x9, x1, x2	0x022084B3
	div x10, x1, x2	0x0220C533
	rem x11, x1, x2	0x0220E5B3
	or x12, x1, x2	0x0020E633
	sll x13, x1, x2	0x002096B3

Nhãn	Mã Assembly	Mã HEX
	lw x14, 4(x3)	0x0041A703
	sw x4, 8(x3)	0x0041A423
	beq x1, x2, tag	0x00208663
end:	addi x5, x0, -15	0xFF100293
	addi x6, x0, 255	0x0FF00313
tag:	add x5, x3, x4	0x004182B3
	sub x7, x5, x3	0x403283B3
	j end	0xFF1FF06F

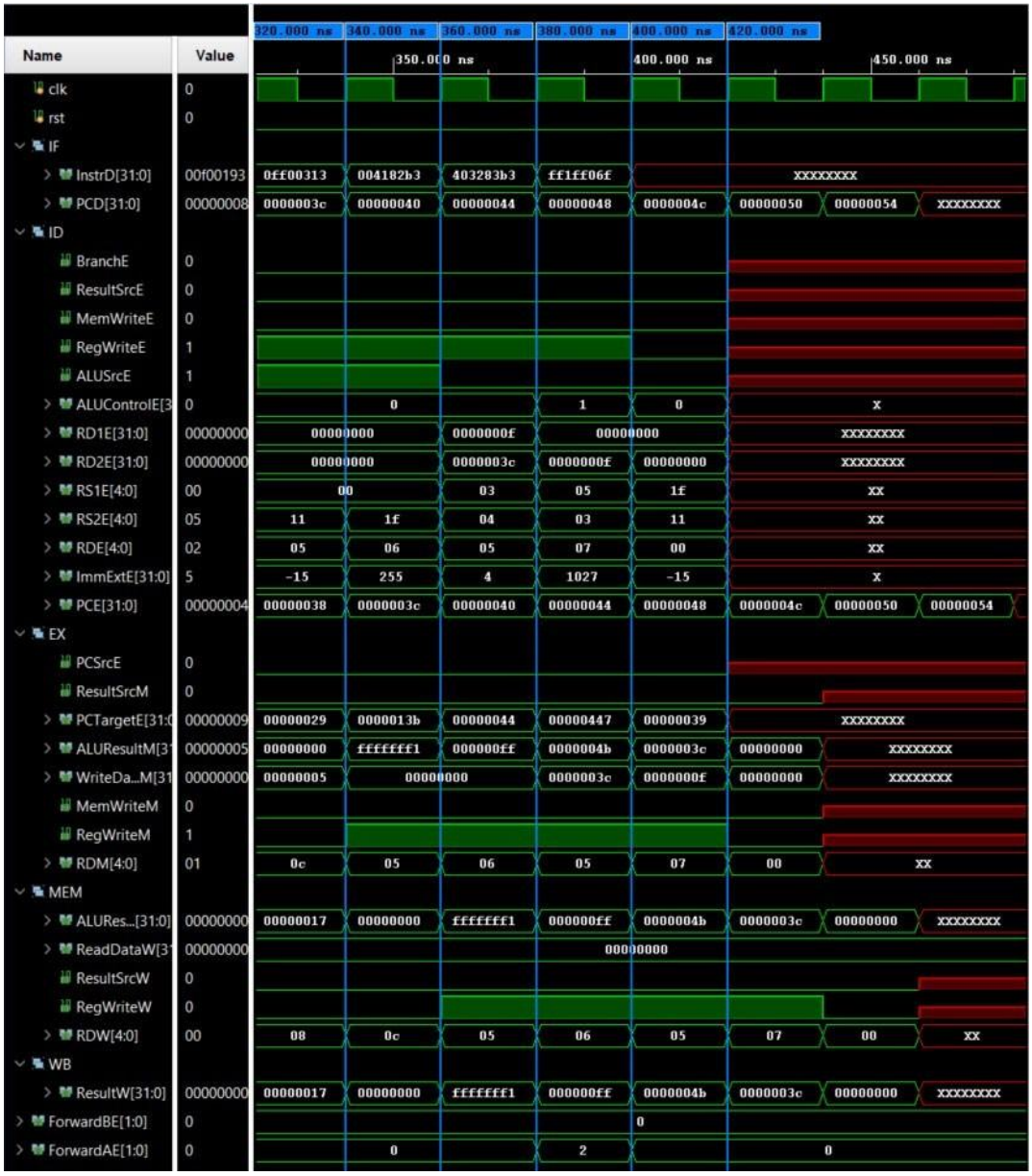
Kết quả mô phỏng dạng sóng ngõ ra của bộ vi xử lý 32 bit sẽ giúp chúng ta hiểu rõ hơn về cách các tín hiệu và dữ liệu di chuyển qua các giai đoạn khác nhau trong quá trình thực thi lệnh. Bằng cách kiểm tra này chúng ta có thể kiểm tra để đảm bảo rằng các lệnh được thực thi chính xác và hiệu quả, cũng như phát hiện và sửa các lỗi có trong việc thiết kế. Kết quả mô phỏng được trình bày dưới đây:



Hình 16: Dạng sóng của bộ vi xử lý 32 bit (từ 0 – 160ns)



Hình 17: Dạng sóng của bộ vi xử lý 32 bit (từ 160 – 320ns)



Hình 18: Dạng sóng của bộ vi xử lý 32 bit (từ 320 – 480ns)

9. KẾT LUẬN

Sau quá trình thực hiện đồ án thiết kế bộ vi xử lý 32 bit dựa trên kiến trúc tập lệnh RISC-V và kiến trúc Harvard, nhóm đã đạt được các yêu cầu sau:

- Thiết kế được một bộ vi xử lý 32 bit
- Mô phỏng được các chức năng của các khối còn trong bộ vi xử lý 32 bit.
- Mô phỏng được toàn bộ hệ thống vi xử lý 32 bit với một số lệnh cơ bản.

10. HƯỚNG PHÁT TRIỂN

Đề tài thực hiện "Thiết kế bộ vi xử lý 32 bit" được nhóm thực hiện đạt được các mục tiêu và yêu cầu thiết kế bạn đầu đề ra nhưng còn hạn chế về khả năng và thời gian thực hiện nên kết quả thực hiện có thể thiếu thiếu sót một vài trường hợp chưa được tối ưu, nhóm đưa ra một số hướng phát triển như sau:

- Mô phỏng thêm nhiều lệnh hơn ở kiến trúc tập lệnh RISC-V.
- Thiết kế bộ vi xử lý với các phép tính số học có dấu chấm động.

Nhúng lên kit FPGA để kiểm tra tín hiệu và chức năng thực tế.